

Lecture 18 - Divide and Conquer

Please see the animations provided with these notes, or check out YouTube as you will find loads of wonderful resources there!

You can also entertain yourself with some Eastern European dancing.

Hungarian Dance - Sorting Algorithms

In this lecture, we continue our look at searching algorithms.

In particular we are going to focus on two famous sorting algorithms:

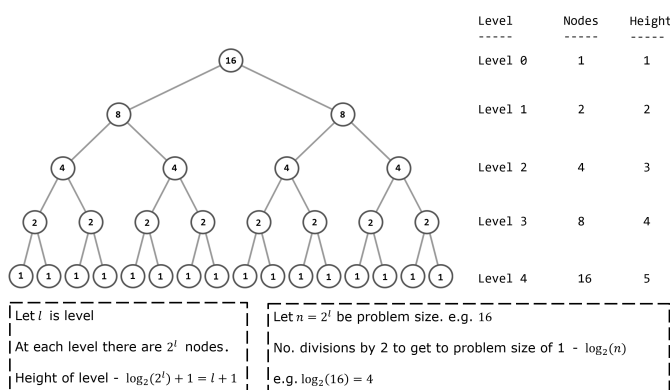
- Merge Sort
- Quicksort.

Both of these are examples of **divide and conquer** algorithms.

Binary Trees, Powers of 2 and Log Base 2

Before we get started it is useful to know a few things about binary trees and logs. We will look at trees properly later in the module.

A perfect binary tree is a tree in which each node has two children and all the leaf nodes (final nodes/last level) are at the same level.



If we call the level l . We can see from the above image that at level 0 there is $2^0 = 1$ node, at level 2 there are $2^2 = 4$ nodes, at level 3 there are 2^3 nodes, ... , at level l there are 2^l nodes.

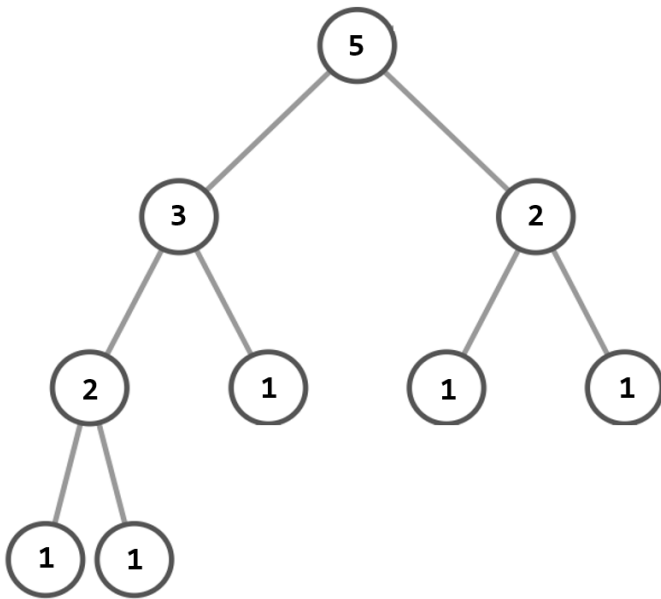
In total we have $\sum_{i=0}^l 2^i = 2^{l+1} - 1$. For example in this perfect binary tree the last level is level 4. Thus we have $2^{4+1} - 1 = 31$ nodes in the tree. Check for yourself!

We can also see that if we have a set of objects of size n (above 16), then we can split this in half a total of 4 times until we get a set of single objects (size 1).

The relationship here is given a problem size of n , we require $\log_2(n)$ splits (divisions of 2).

Note that if our problem size is not a power of 2, e.g. 1, 2, 4, 8, 16, ... then the maximum number of splits to get to a subproblem of size 1 will be $\lfloor \log_2(n) \rfloor + 1$. The weird symbols $\lfloor$ and $\rfloor$ mean floor the number. e.g. $\lfloor 6.34 \rfloor = 6$.

e.g for 5 items, we have a max split of $\lfloor \log_2(5) \rfloor + 1 = 3$.



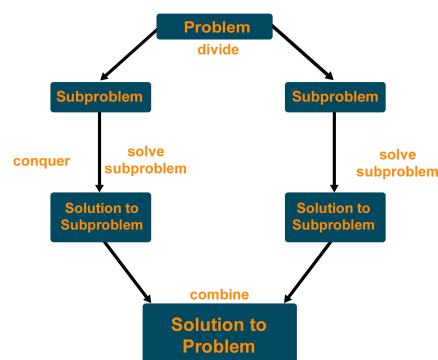
This is an example of a full binary tree.

Divide and Conquer

Divide and conquer algorithms take the following [approach](#):

1. **Divide** - Divide the problem into smaller subproblems of the same structure using recursion
2. **Conquer** - Conquer the subproblems when they are small enough (base case)
3. **Combine** - Combine the solutions of the subproblems to solve the original problem.

It is quite normal to view **Conquer** and **Combine** as a single stage as they work closely together.



Merge Sort

Merge Sort is a **sorting** algorithm that will sort a list of n items.

It is a divide and conquer algorithm that works as follows

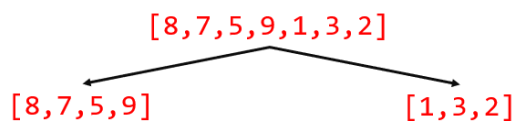
1. Divide the unsorted list into n sublists, each containing one item (a list of one element is item sorted).
2. Repeatedly merge sublists to produce new sorted sublists until there is only one sublist remaining. This will be the sorted list.

The following series of figures show how this is done.

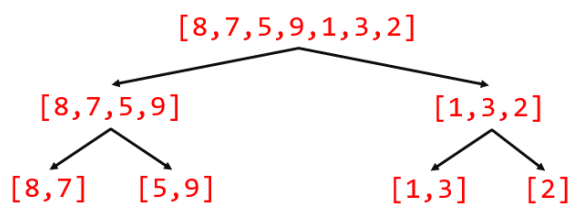
Please note that in practice we don't evaluate it this way; however, it is easier to visualise. In practice we keep evaluating the left half first (depth-first search).

[8,7,5,9,1,3,2]

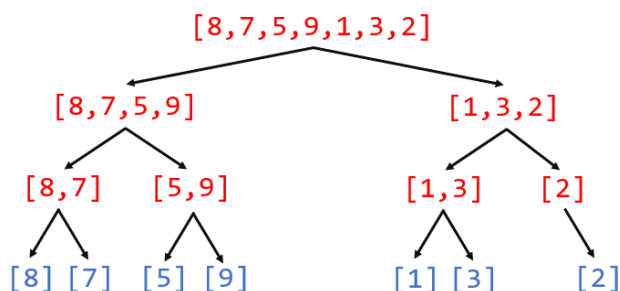
List to be sorted



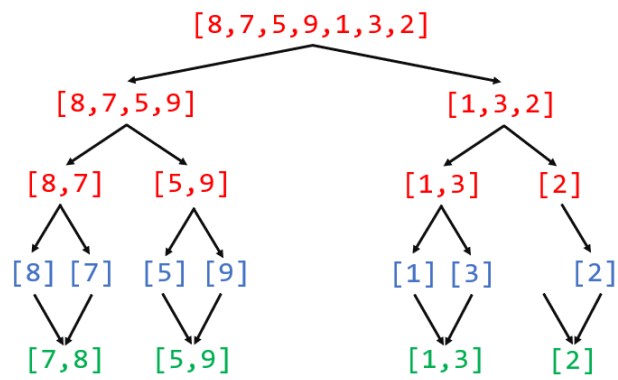
Divide list into left and right lists



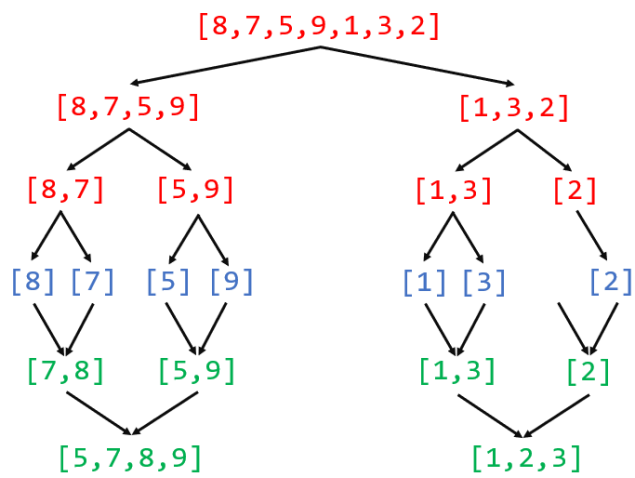
Divide these into left and right lists



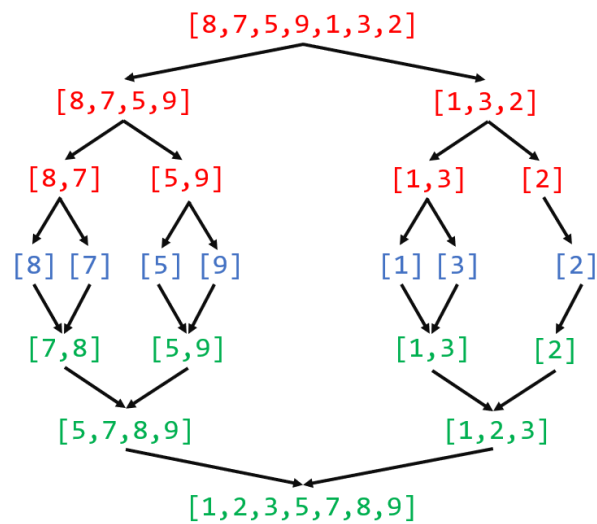
Divide these into left and right lists, each contain a single item



Combine sublists into sorted list



Combine sublists into sorted list



Combine sublists into a fully sorted list

Python Code

The code is structured as follows:

```
# This is just pseudocode demonstrating the overall structure

def merge(left, right):
    """ merges the left and right lists into one """
    ... # code to do this
    return merged

def merge_sort(items)
    """ Sorts a list of items """
    if len(items) <= 1: # base case
        return items

    left = ... # get the left half of items
    right = ... # get the right half of items

    left = merge_sort(left) # recursively divide left half
    right = merge_sort(right) # recursively divide right half

    merged = merge(left, right) # merge left and right lists

    return merged

list1 = [5,4,3,6,2]
merge_sort(list1) # returns [2,3,4,5,6]
```

We have extrapolated the functionality to merge two lists into the function `merge()`. This then makes `merge_sort` very clean.

It has 4 steps:

1. Split `items` into `left` and `right` sublists.
2. Recursively split `left` sublist.
3. Recursively split `right` sublist.
4. Merge `left` and `right` sublists.

You should view the merge sort animation with code to see this working

Below is a full implementation of this in Python.

```
def merge_sort(items):
    """A recursive merge sort algorithm"""

    # Base case for recursion, stop when the list has been divided into a single
    item
    if len(items) <= 1:
```

```

        return items
    midpoint = (len(items)-1) // 2 # Calculate the midpoint index
    left = items[0:midpoint+1] # Create left half list
    right = items[midpoint+1:len(items)] # Create right half list

    left = merge_sort(left) # Recursively divide left half
    right = merge_sort(right) # Recursively divide right half

    merged = merge(left, right) # merge left and right lists

    return merged

def merge(left, right):
    """Merges the items in left and right into a new ordered list called merged"""
    merged = [] # New list for merging the items
    index_left = 0 # left current position
    index_right = 0 # right current position

    # While there are still items to merge
    while index_left < len(left) and index_right < len(right):

        # Find the lowest of the two items being compared
        if left[index_left] < right[index_right]:
            merged.append(left[index_left])
            index_left += 1
        else:
            merged.append(right[index_right])
            index_right += 1

    # Add to the merged list any remaining data from left list or right list
    if index_left < len(left):
        merged.extend(left[index_left:])
    else:
        merged.extend(right[index_right:])

    return merged

list1 = [5,4,3,6,2]
merge_sort(list1) # returns [2,3,4,5,6]

```

Time Complexity

The time complexity of a merge sort is $O(n \log(n))$. To calculate time complexity, you need to consider the two main operations:

- Splitting an original list with n items until you have n sublists of single items
- Merging n sublists until you have one ordered list of n items

Splitting

- After 1 split there will be 2 sublists with $\frac{n}{2}$ items each

- After 2 splits there will be 4 sublists with $\frac{n}{4}$ items each
- After 3 splits there will be 8 sublists with $\frac{n}{8}$ items each
- ...
- After $\log_2(n)$ splits there will be n sublists with 1 item each

Therefore, the time complexity of splitting is $O(\log(n))$, as the list is divided $\log(n)$ times (where n is the number of items in the list).

Merging

During the merge process, the sublists are combined by examining every item in turn and placing it in order into a merged list (i.e. **merged** in the pseudocode). This means that, in the worst-case scenario, there will be n comparisons to order all n items for each merge.

Note that, depending on the value of the items in the sublists, sometimes not all n items will need to be compared. Once all of the items from one sublist have been placed, the remaining items in the other sublist can just be added to the end of the merged list, because they are already in order. However, this does not affect the overall time complexity as, in the worst case, all n items must be compared.

If we have two lists of size n , then merging them requires at most $2n - 1$ comparisons.

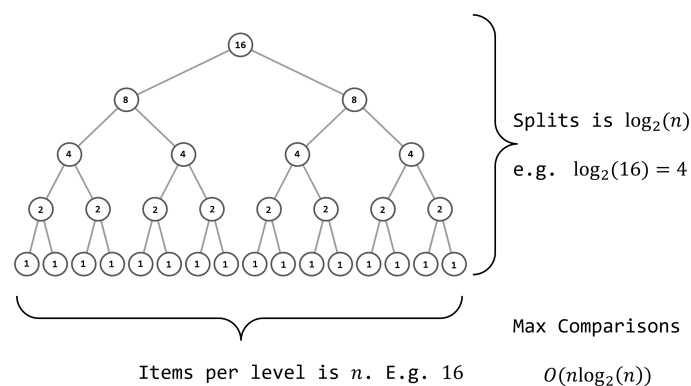
This is $O(n)$.

Splitting and merging combined

- We split the list into $\log(n)$ levels. We need to recombine these.
- Each level of the tree requires $O(n)$ comparisons to merge.
- There are $\log(n)$ levels.

Therefore, we perform $O(n)$ comparisons on $\log(n)$ levels of the tree.

Thus we have a time complexity of $O(n \log(n))$.



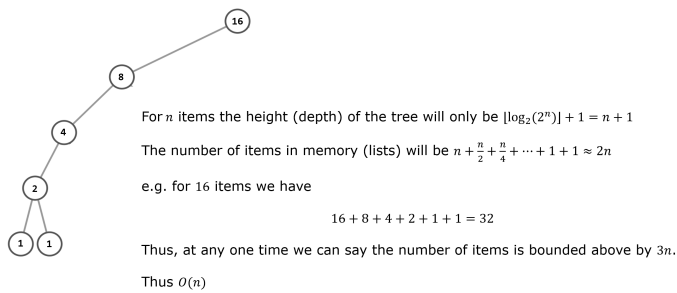
Best, average, and worst-case time complexity

Note that the complexity $O(n \log(n))$ is the same in all three cases (worst, best, and average) as the full split and merge process is always executed.

Space Complexity

You might think $O(n \log(n))$ as we have n items at each level.

However, the tree only has a portion of it in memory at one time due to the stack calls.



Therefore, the space complexity of the algorithm is $O(n)$.

Quicksort

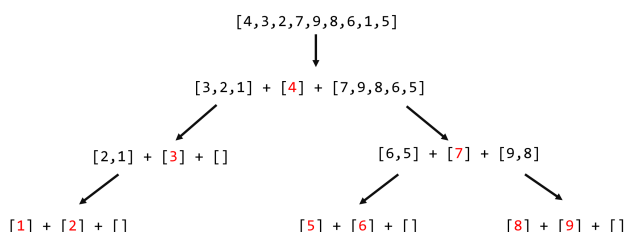
Quicksort is a **sorting** algorithm that will sort a list of n items.

It is a divide and conquer algorithm that works as follows

1. Choose a pivot item from the list and divide the list into two new lists
 - items less than and equal to the pivot are placed on the left side
 - items greater than the pivot are placed on the right.
2. We repeat the above process for the resulting left and right lists until we end up with a list of size 1.
 - At this stage, all elements are already sorted, and we combine all the results to return the final sorted list.

The following figure visualises how this is done. Note that when programmed only parts of this tree will be in memory at one time.

I highly recommend that you check out the animations on the module to see how it works.



Notice that all the pivots (in red) are
in sorted order from left to right!
These are then combined back together

Not in-place

Python Code

You should see the animation on the module and paste this into [Python Tutor](#) to help understand how it works.

```
def quicksort(items):
    if len(items) <= 1: # base case - single item in list
        return items

    pivot = items[0] # choose pivot as first item
    low = []
    high = []

    # sort the items into low or high list
    for i in range(1, len(items)):
        if items[i] < pivot:
            low.append(items[i])
        else:
            high.append(items[i])

    # recursively call quicksort on lower and higher
    # return combined result
    return quicksort(low) + [pivot] + quicksort(high)

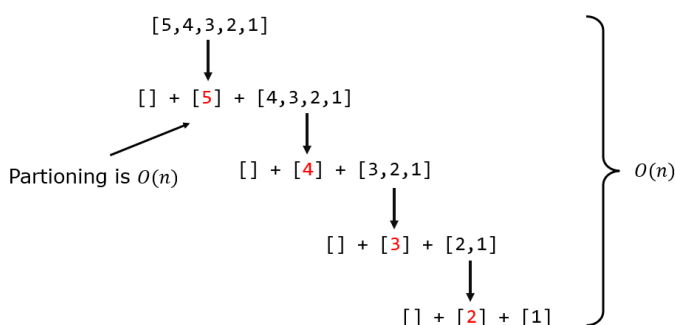
list1 = [4,3,2,7,9,8,6,1,5]
quicksort(list1)
```

Time Complexity

The best and average time complexity are the same as merge sort and you can analyse them in a similar way.

However, worst-case time complexity for quicksort is $O(n^2)$. We can see this via the following diagram.

If the items are in completely the wrong order and we choose a bad pivot, here, the first item then we get the following.



It takes $O(n)$ time to partition a list into lower and higher lists and we do $O(n)$ splits.

Thus we have $O(n^2)$.

Space Complexity

The space complexity is $O(n^2)$. If you look at the previous figure you will note that if we have n items, in the worst-case we get a horribly unbalanced tree. At each level there are approximately $n, n-1, n-2, n-3, \dots, 1$. If we add these up we get $\frac{n^2}{2} + \frac{n}{2}$.

This results in a worst-case space complexity of $O(n^2)$.

In-place

Python Code

You should see the animation on the module and paste this into [Python Tutor](#) to help understand how it works.

```
def swap(items, i, j):
    """ Swaps item i with item j """
    temp = items[i]
    items[i] = items[j]
    items[j] = temp

def partition(items, start, end):
    """ Pivots """
    pivot = items[end]
    i = start - 1
    for j in range(start, end):
        if items[j] <= pivot:
            i += 1
            swap(items, i, j)

    swap(items, i+1, end)
    return i + 1

def quick_sort(items, start, end):
    if start < end:
        q = partition(items, start, end)
        quick_sort(items, start, q - 1)
        quick_sort(items, q+1, end)

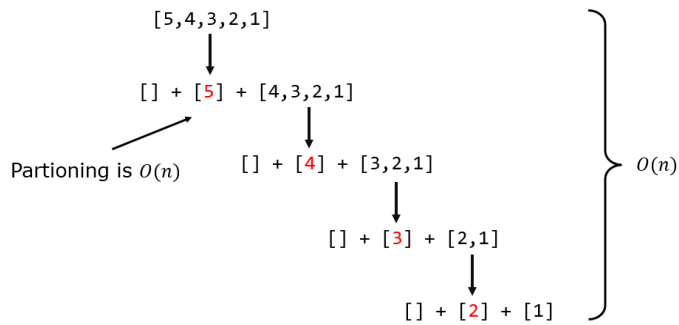
if __name__ == "__main__":
    items = [4,3,2,5,6,4,2,1]
    quick_sort(items, 0, len(items)-1)
```

Time Complexity

This is the same as the not in-place version and follows the same analysis.

Space Complexity

The improvement here is that do not make copies of the original list. The only thing we store are pointers to `start`, `end` and `q` (final position of pivot) for each split (stack frame). We know from previous inspection of the following figure that the worst-case for the number of stack frames is $O(n)$.



Thus the space complexity is $O(n)$.

Note that it is possible with some very clever optimisations to get this down to $O(\log(n))$. I've provided a link to the original [paper](#).

Summary

- Merge sort and quicksort are comparison sorts
- Examples of **Divide and Conquer**
- Both are comparison sorting methods

Time Complexity	Merge Sort	Quicksort (Not in-place)	Quicksort (In-place)
Best-case	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Average-case	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Worst-case	$O(n \log(n))$	$O(n^2)$	$O(n^2)$
Space Complexity	$O(n)$	$O(n^2)$	$O(n)$

Comparison Sorting Summary Table

Here is a table of all the comparison sort algorithms we have looked at.

Time Complexity	Bubble Sort	Insertion Sort	Merge Sort	Quicksort (Not in-place)	Quicksort (In-place)
Best-case	$O(n)$	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Average-case	$O(n^2)$	$O(n^2)$	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
Worst-case	$O(n^2)$	$O(n^2)$	$O(n \log(n))$	$O(n^2)$	$O(n^2)$
Space Complexity	$O(1)$	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n)$

There are a lot more sorting methods and optimisations of these. Python uses **Timsort**.